



UNIVERSITY OF ESWATINI

Department of Computer Science

SOFTWARE PROJECT PLAN

for the

Smart Gate Access Control System

University of Eswatini: Campus Safety & Access Management

Section	Page
Introduction	5
Background	5
Project Aims	5
Project Objectives	5
Primary Objectives	5
Secondary Objectives	6
Persons Responsible	6
Project Summary	6
User and Client Involvement	7
Identified Stakeholders	7
Client Responsibilities	7
Risk Identification and Management	9
Standards, Guidelines, and Procedures	9
Coding Standards	9
Version Control Procedures	9
Documentation Standards	10
Meeting and Communication Protocols	10

Section	Page
Organization of the Project	11
Project Team Structure and Roles	11
Project Phases and Schedule	13
Task Description and Dependency Table	13
Gantt Chart	15
PERT Analysis	16
Requirements Analysis and Design	18
Functional Requirements	18
Non-Functional Requirements	19
System Design Overview	20
Use Case Diagram (Figure 1)	21
Class Diagram (Figure 2)	22
Sequence Diagram	23
Implementation	17
Technology Stack Justification	17
Database Schema	18
Implementation Environment	20

Section	Page
Testing	23
Testing Strategy	23
Unit Testing	23
Integration Testing	23
System Testing	23
User Acceptance Testing (UAT)	23
Testing Schedule	24
Resources	24
Quality Assurance	25
Change Management	26
Change Control Process	27
Configuration Management	27
References	28

1. Introduction

The Smart Gate Access Control System is a software-driven access management solution designed and developed for the University of Eswatini (UNESWA). The primary purpose of this document is to present a comprehensive software development project plan that describes all activities, resources, schedules, risks, and quality assurance measures associated with the development and deployment of the **Smart Gate system**.

1.1. Background

The University of Eswatini is the principal institution of higher learning in the Kingdom of Eswatini, accommodating thousands of registered students, academic staff, and administrative personnel across its main campus at Kwaluseni and satellite campuses. Campus security and the controlled movement of persons and property across institutional gates constitute a fundamental operational responsibility for university management.

1.2. Project Aims

The main aims of the Smart Gate Access Control System are:

- a. To design and develop a Java-based secure and efficient access control system⁸ that uses bar code card scanning for managing campus entry and exit.
- b. To reduce security risks associated with manual gate operations, and to improve overall campus safety and accountability through digital tracking.

1.3. Project Objectives

Primary Objectives

- a. To design and develop a smart gate monitoring system that facilitates secure, card-enabled access control at campus gate points for all registered students and staff members of UNESWA.
- b. To implement a MySQL-backed relational database that records, in real time, all gate entry and exit events, including the user identity, timestamp, gate point, and action type.

- c. To provide an administrative dashboard that enables authorised university administrators to manage user accounts, view gate activity logs, and generate analytical reports.
- d. To integrate Z-Xing (Zebra Crossing) barcode and QR code scanning technology to support automated identity verification via student and staff identification cards.
- e. To replace the current manual, guard-only gate management process with a hybrid model in which guards operate scanner-equipped workstations and the system maintains a complete digital audit trail.

Secondary Objectives

- a. To provide role-based access control (RBAC) distinguishing between administrator, staff, and student user profiles with appropriate system privileges.
- b. To generate statistical reports on daily, weekly, and per-gate activity to support university decision-making on security resource deployment.
- c. To ensure that the system adheres to acceptable performance benchmarks, including a card-scan-to-gate-response time of fewer than two seconds under standard operating conditions.

1.4. Persons Responsible

The successful development and implementation of this project involves a structured team with clearly defined roles:

- a. **Project Manager** – Responsible for planning, coordination, and communication with stakeholders.
- b. **System Analyst** – Gathers and documents system requirements.
- c. **UI/UX Developer** – Designs and develops the user interface.
- d. **Backend Developer** – Implements system logic, designs and manages the database and manage database interaction.
- e. **Hardware/Integration Engineer** – Handles scanner integration and hardware setup.
- f. **QA/Test Engineer** – Ensures system quality through testing and documentation.

1.5. Project Summary

This project focuses on the design and implementation of a Java-based desktop application integrated with a MySQL database to manage gate access. The system enables users to scan their identification cards at entry and exit points, automatically capturing essential data such as user identity, time, and gate location.

Additionally, the system provides an administrative dashboard for monitoring gate activity, managing users, and generating reports. The Smart Gate system enhances accountability, improves safety, and supports better decision-making by providing accurate and real-time data on campus movement.

2. User and Client Involvement

The successful development of Smart Gate is fully relying on efficient collaboration between the development team and the primary stakeholder groups within UNESWA. The following parties are identified as key participants in the project:

2.1. Identified Stakeholders

- a. **University Administration:** Provides strategic direction, project approval, and sign-off on functional and non-functional requirements. Administration will also facilitate access to campus infrastructure for testing and deployment.
- b. **Security Department / Guards:** The primary end-users of the gate scanning interface. Guards will participate in requirements workshops, usability testing, and training sessions. Their operational knowledge is essential for capturing practical workflow requirements.
- c. **Registered Students:** Users of the card-scanning entry mechanism. Student representatives will participate in User Acceptance Testing (UAT) to validate that the card scanning and entry process is efficient and user-friendly.
- d. **Academic and Administrative Staff:** Staff members holding university-issued identification cards. They will participate in UAT and provide feedback on the entry/exit experience.
- e. **IT Department:** Responsible for providing and maintaining the server infrastructure on which the MySQL database will be hosted. The IT department will also advise on network configuration and security policies.

2.2. Client Responsibilities

The university, as a client, is expected to provide the following resources and services to support the implementation of this project:

- a. Physical access to designated gate points for systems integration and testing.
- b. Sample (anonymized) student and staff data for database population during development.
- c. Barcode card scanner hardware at the agreed gate workstations.

- d. Server environment provisioned with MySQL 8.x and Java Runtime Environment (JRE 11+).

3. Risk Identification and Management

Risk management is an essential component of responsible software project planning. The following risks have been identified through structured analysis of the project's technical, organizational, and environmental context. For each risk, a likelihood rating, impact severity, and mitigation strategy have been assigned.

#	Risk	Likelihood	Impact	Mitigation Strategy
1	Hardware scanner malfunction or incompatibility	Medium	High	Procure tested Z-Xing-compatible barcode scanners; maintain spare units; implement manual ID entry fall back.
2	Database connection failure at gate terminals	Low	High	Implement local caching and offline mode; scheduled DB health checks; JDBC connection pooling.
3	Unauthorised access / credential sharing	Medium	High	RBAC enforcement; session timeout; audit trails logged in gate_activity table; encrypted password storage.
4	Scope creep from stakeholder requests	High	Medium	Formal change control process; document baseline requirements; weekly sign-off meetings with supervisor.
5	Team member unavailability	Medium	Medium	Cross-training; maintain shared code repository on GitHub; pair-programming for critical modules.
6	MySQL version incompatibility	Low	Medium	Lock MySQL version in project documentation; test on target server OS before deployment.

#	Risk	Likelihood	Impact	Mitigation Strategy
7	Poor user adoption by staff/students	Medium	Medium	Run training workshops; provide user manual; phased rollout with support desk availability.
8	Data loss or corruption	Low	High	Automated daily backups; transaction-safe SQL; prepared statements prevent accidental data corruption.

4. Standards, Guidelines, and Procedures

All team members engaged in the Smart Gate project are required to adhere to the following standards, guidelines, and procedural norms throughout the software development lifecycle:

4.1. Coding Standards

- Java code shall conform to the Oracle Java Code Conventions (2020 revision), including consistent indentation (4 spaces), meaningful variable naming, and Javadoc comments on all public methods and classes.
- All SQL queries shall use parameterized PreparedStatements via JDBC to prevent SQL injection vulnerabilities. Raw string concatenation in SQL is strictly prohibited.
- UI component classes (like Min LandingFrame, LoginFrame and DashboardFrame) shall be separated from data-access classes (such as DBConnection) in accordance with the Model-View-Controller (MVC) architectural pattern.

4.2. Version Control Procedures

- All source code shall be maintained in a Git repository hosted on GitHub. Commits must include descriptive messages referencing the relevant task identifier.
- No direct commits to the main branch are permitted. All changes must be submitted via feature branches and reviewed through pull requests before merging.

4.3. Documentation Standards

- Project documentation shall be produced in Microsoft Word format, using Arial 12pt body text, appropriate heading hierarchy, and a consistent colour scheme aligned with the UNESWA institutional palette.
- All diagrams (PERT, Gantt, UML use-case, class, and sequence diagrams) shall be produced using a recognized CASE tool such as Visual Paradigm or Lucid chart and exported as high-resolution images for inclusion in project documents.

4.4. Meeting and Communication Protocols

- Weekly project meetings shall be held every Monday at 10:00 AM. Minutes shall be recorded by a rotating secretary and distributed to all team members and the supervisor within 24 hours.
- All formal communication with the client (university administration) shall be channelled through the project manager, who is the designated point of contact.

5. Organization of the Project

The Smart Gate project is an independent, standalone academic software development project under the oversight of the Department of Computer Science at the University of Eswatini. The project does not have formal dependencies on other concurrent projects; however, it has an interface relationship with the university's existing student and staff record systems insofar as identity data will be synchronized for use within the Smart Gate database.

5.1. Project Team Structure and Roles

The following table describes the organizational structure of the Smart Gate development team, including the roles, responsibilities, and skills associated with each position:

Role	Responsibilities	Skills Required
Project Manager	Plans, schedules and controls the project; chairs weekly team meetings; liaises with university stakeholders; manages risks and change requests.	<i>Leadership, MS Project / Gantt, communication</i>

Role	Responsibilities	Skills Required
System Analyst	Elicits and documents requirements from guards, admin staff, students; produces use-case diagrams and system specifications.	<i>Requirements engineering, UML, interviewing</i>
UI/UX Developer	Designs and implements the Java Swing interface; builds custom JPanel components, themed login screens and dashboard widgets.	<i>Java Swing, OOP, UI design principles</i>
Backend Developer	Implements business logic (access control rules, session management, RBAC); writes JDBC data-access layer.	<i>Java SE, JDBC, SQL, OOP design patterns</i>
Database Administrator	Designs MySQL schema (users, cards, gate_activity tables); writes optimised queries; manages backups and DB security.	<i>MySQL, SQL, database normalisation</i>
Hardware / Integration Engineer	Integrates ZXing barcode library with USB card scanners; tests scanning accuracy and latency; handles device driver configuration.	<i>ZXing, Java I/O, hardware interfaces</i>
QA / Test Engineer	Designs and executes unit, integration and user-acceptance tests; documents defects; verifies security controls.	<i>JUnit, test planning, regression testing</i>
Technical Writer	Produces user manual, installation guide, and this project plan document; prepares training materials.	<i>Technical writing, MS Word, documentation tools</i>

6. Project Phases and Schedule

The Smart Gate project adopts an iterative (Agile) and incremental software development lifecycle model, drawing on elements of the Unified Process. This model was selected because it accommodates evolving user requirements, supports incremental delivery of working software components, and provides structured milestones for progress monitoring. The project is

organized into ten distinct tasks (A through J), spanning a total of 36 working days (approximately ten weeks).

6.1. Task Description and Dependency Table

The following table presents a comprehensive breakdown of each project task, including estimated duration, predecessor dependencies, assigned responsibilities, and estimated cost per task in Swazi Lilangeni (SZL):

Task	Activity Description	Duration (days)	Predecessor	Responsible	Est. Cost (SZL)
A	Requirements Analysis & Stakeholder Interviews	5	—	Project Manager, Analyst	3,500
B	System Architecture & Database Design	4	A	System Architect, DBA	2,800
C	UI/UX Design — Java Swing & JPanel Layouts	5	B	UI Developer	2,200
D	Backend Development — Java Business Logic	10	B	Backend Developer	6,500
E	Database Implementation — MySQL Schema & Queries	6	B	DBA, Backend Dev.	3,000
F	Card Scanner Integration — ZXing Library	5	D, E	Hardware/Integration Dev.	4,000
G	Admin Dashboard — Reports & Analytics Module	6	C, D	Full-Stack Developer	3,800
H	System Integration & Module Testing	5	F, G	QA Engineer, Dev. Team	2,500
I	User Acceptance Testing — Staff & Students	4	H	QA Engineer, End Users	1,500

Task	Activity Description	Duration (days)	Predecessor	Responsible	Est. Cost (SZL)
J	Documentation, Training & Deployment	3	I	All Team Members	1,200

The **critical path** for the Smart Gate project is: $A \rightarrow B \rightarrow D \rightarrow F \rightarrow H \rightarrow I \rightarrow J$, representing the sequence of tasks that directly determines the minimum project completion time.

Any delay on the critical path will result in a corresponding delay to the project delivery date.

6.2. Gantt Chart for Smart Gate Project Schedule

The Gantt chart below provides a visual representation of the project schedule across a ten-week period. Each task is mapped onto the relevant week(s) during which it is planned for execution. Shaded cells indicate active task periods. Tasks on the critical path (A, B, D, F, H, I, J) are shown in darker shading.

TASK ID	Activity	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8	Wk 9	Wk 10
A	Requirements Analysis										
B	Architecture & DB Design										
C	UI/UX Design										
D	Backend Development										
E	MySQL Implementation										
F	Z-Xing API Card Integration										
G	Admin Dashboard										
H	Integration & Testing										
I	User Acceptance Testing										

J	Documentations, Training & Deploy										
---	--------------------------------------	--	--	--	--	--	--	--	--	--	--

6.3. PERT Analysis: Earliest and Latest Start Times

The Program Evaluation and Review Technique (PERT) is a tool that provides a more rigorous analytical framework than the Gantt chart alone, enabling the calculation of Earliest Start Times (EST), Latest Start Times (LST), and Slack Times for each project task. These values are critical for identifying the project's critical path and determining the scheduling flexibility available for non-critical tasks.

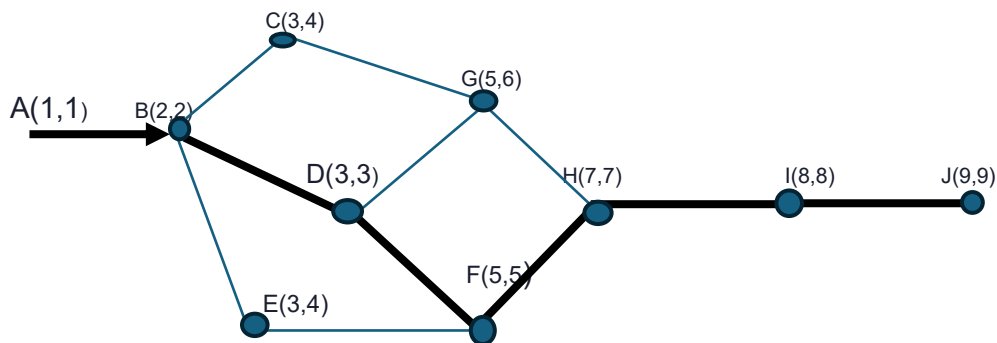
Definition of PERT Terms:

- a) **Earliest Start Time (EST):** The earliest point in the project schedule at which a task may begin, given that all predecessor tasks have been completed.
- b) **Latest Start Time (LST):** The latest point at which a task may begin without causing a delay to the overall project completion date.
- c) **Slack Time:** The difference between LST and EST. A slack time of zero indicates that the task lies on the critical path.
- d) **Earliest Completion Time (ECT):** The earliest date on which the entire project may be completed, assuming all tasks are executed within their allocated durations. For Smart Gate, the ECT is 36 working days.
- e) **Critical Path:** The sequence of tasks from project start to project end for which all slack times are zero. For Smart Gate, the critical path is: A → B → D → F → H → I → J.

Task	Activity	EST (days)	LST (days)	Slack Time	Duration	On Critical Path?
A	Requirements Analysis	0	0	0	5	YES

Task	Activity	EST (days)	LST (days)	Slack Time	Duration	On Critical Path?
B	Architecture & DB Design	5	5	0	4	YES
C	UI/UX Design	9	12	3	5	No
D	Backend Development	9	9	0	10	YES
E	MySQL Implementation	9	10	1	6	No
F	ZXing Card Integration	19	19	0	5	YES
G	Admin Dashboard	15	17	2	6	No
H	Integration & Testing	24	24	0	5	YES
I	User Acceptance Testing	29	29	0	4	YES
J	Docs, Training & Deploy	33	33	0	3	YES

A full graphical PERT network diagram.



7. Requirements Analysis and Design

7.1. Functional Requirements

The functional requirements of the Smart Gate system have been elicited through stakeholder interviews with university security staff, administrative officers, and a sample of registered students. The following functional requirements are baselined for this project:

- a) **FR-01:** The system shall enable registered students and staff to gain access to campus by presenting their university-issued identification card to a ZXing-compatible USB barcode or QR code scanner at the gate.
- b) **FR-02:** The system shall record each gate event (entry or exit) in the gate_activity database table, capturing the user ID, user full name, role, action type (ENTRY/EXIT), gate identifier, and timestamp.
- c) **FR-03:** The system shall provide a secure login interface for administrators, accepting a username/ID number and password, validated against the users table in the MySQL database.
- d) **FR-04:** The administrator dashboard shall display a summary of current campus statistics, including total entries, total exits, users currently on campus, and unique users for the current day.
- e) **FR-05:** The administrator shall be able to register new users (students, staff) and associate them with a physical card UID in the cards table.
- f) **FR-06:** The system shall support manual entry of an ID number as a fallback mechanism in cases where the card scanner is unavailable.
- g) **FR-07:** The system shall generate tabular reports segmented by day, gate point, and user role, accessible from the Reports panel of the administrator dashboard.
- h) **FR-08:** The system shall support a Settings panel allowing the administrator to configure database connection parameters and institutional information.

7.2. Non-Functional Requirements

- a) **NFR-01 Performance:** Card scan-to-system-response time shall not exceed two seconds under normal operating conditions (network latency < 50 ms, < 100 concurrent users).
- b) **NFR-02 Security:** Passwords shall be encrypted, stored as salted SHA-256 hashes. All SQL interactions shall use PreparedStatements. Session tokens shall expire after a configurable inactivity period.
- c) **NFR-03 Usability:** The user interface shall be operable by security guards with basic computer literacy, requiring no more than 15 minutes of training to perform standard scanning operations.
- d) **NFR-04 Reliability:** The system shall maintain an uptime of at least 99% during university operating hours (07:00–22:00), with graceful error handling for database connection failures.
- e) **NFR-05 Maintainability:** Source code shall be modular, with a clear separation between UI components (Swing), business logic, and data access layers to facilitate future modifications.

7.3. System Design Overview

The Smart Gate system employs a three-tier architectural pattern comprising a Presentation Layer (Java Swing GUI), a Business Logic Layer (Java classes handling access control, session management, and RBAC), and a Data Access Layer (JDBC-mediated interaction with MySQL 8.x). The following key design decisions have been made:

- a) The main **GateActivity panel** supports real-time manual or scanned entry logging, with PreparedStatement-based inserts into the gate_activity table ensuring both data integrity and SQL injection prevention.
- b) The **LoginFrame** class implements administrator authentication, validating credentials against the users table and redirecting authorised users to the DashboardFrame.
- c) The **DashboardFrame** class provides a sidebar navigation panel with links to the Dashboard summary, Gate Activity log, User Management, Reports, and Settings panels.

- d) The Theme class defines a centralised colour palette and font configuration to ensure visual consistency across all UI components, aligned with a professional dark navy and purple colour scheme.
- e) The **DBConnection** class provides a static getConnection() factory method encapsulating all MySQL connection parameters, enabling consistent connection management across all data-access components.

UML diagrams for the Smart Gate Control System are shown below.

USE CASE DIAGRAM

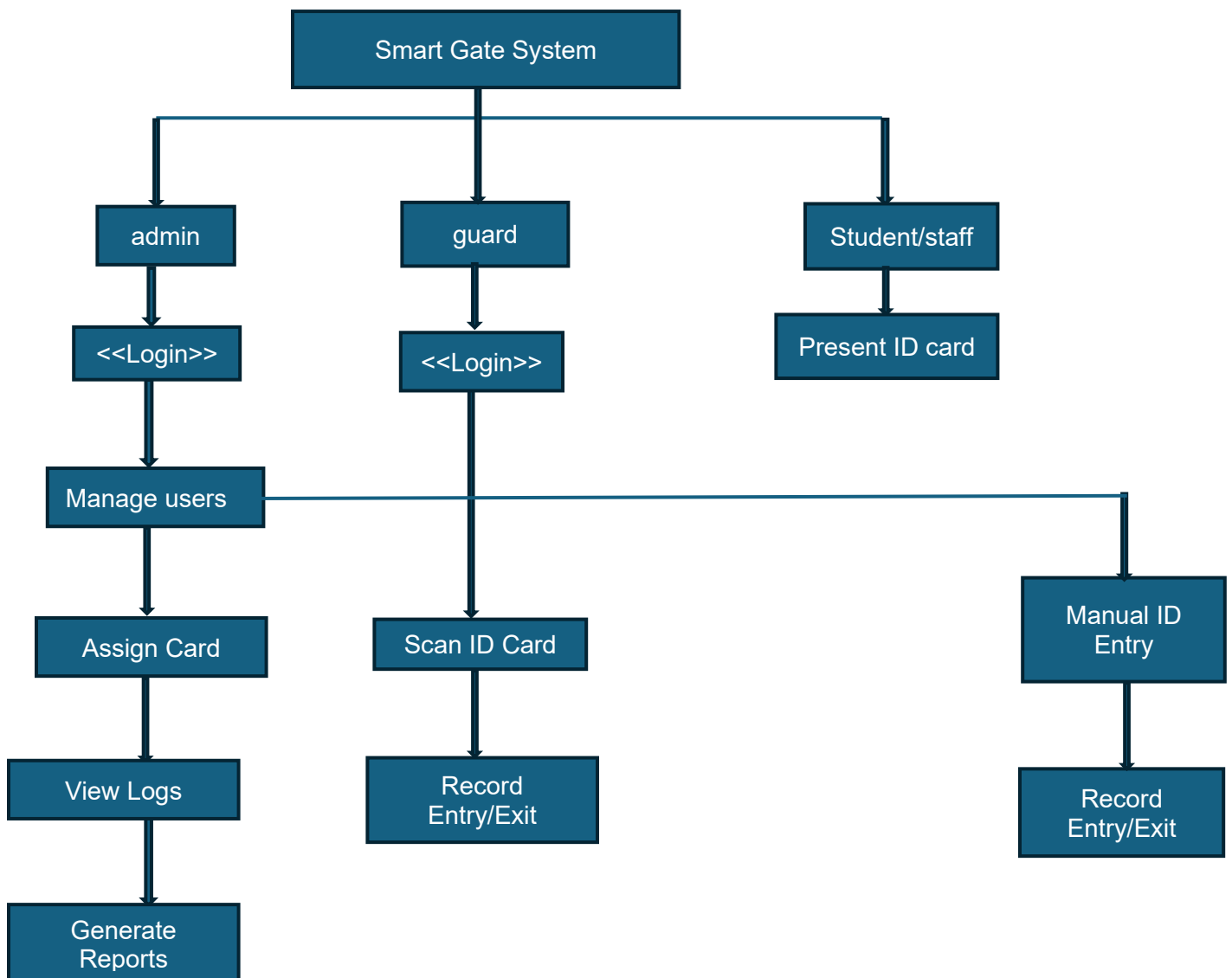


Figure 1:use case diagram for Smart Gate Access Control System

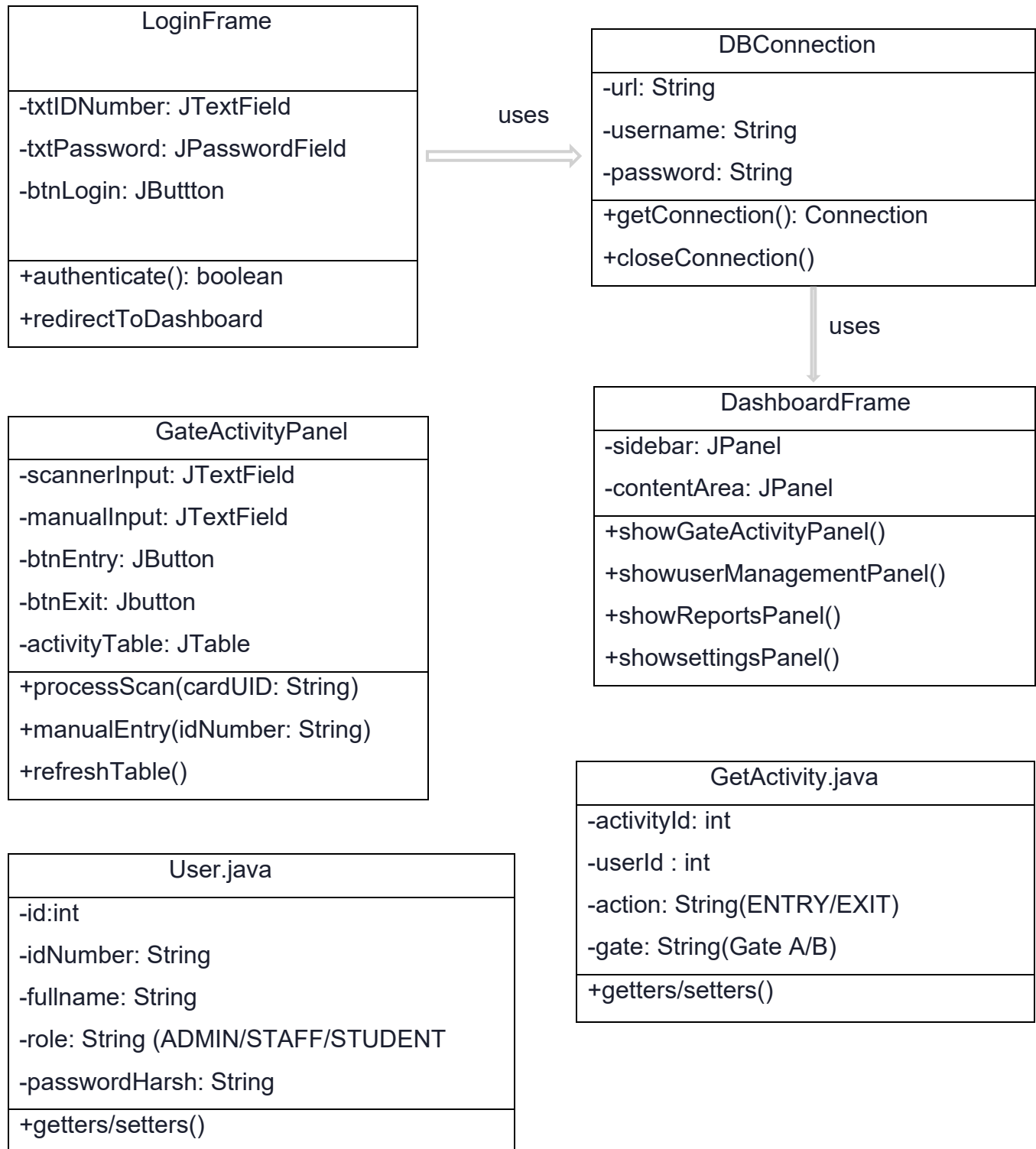
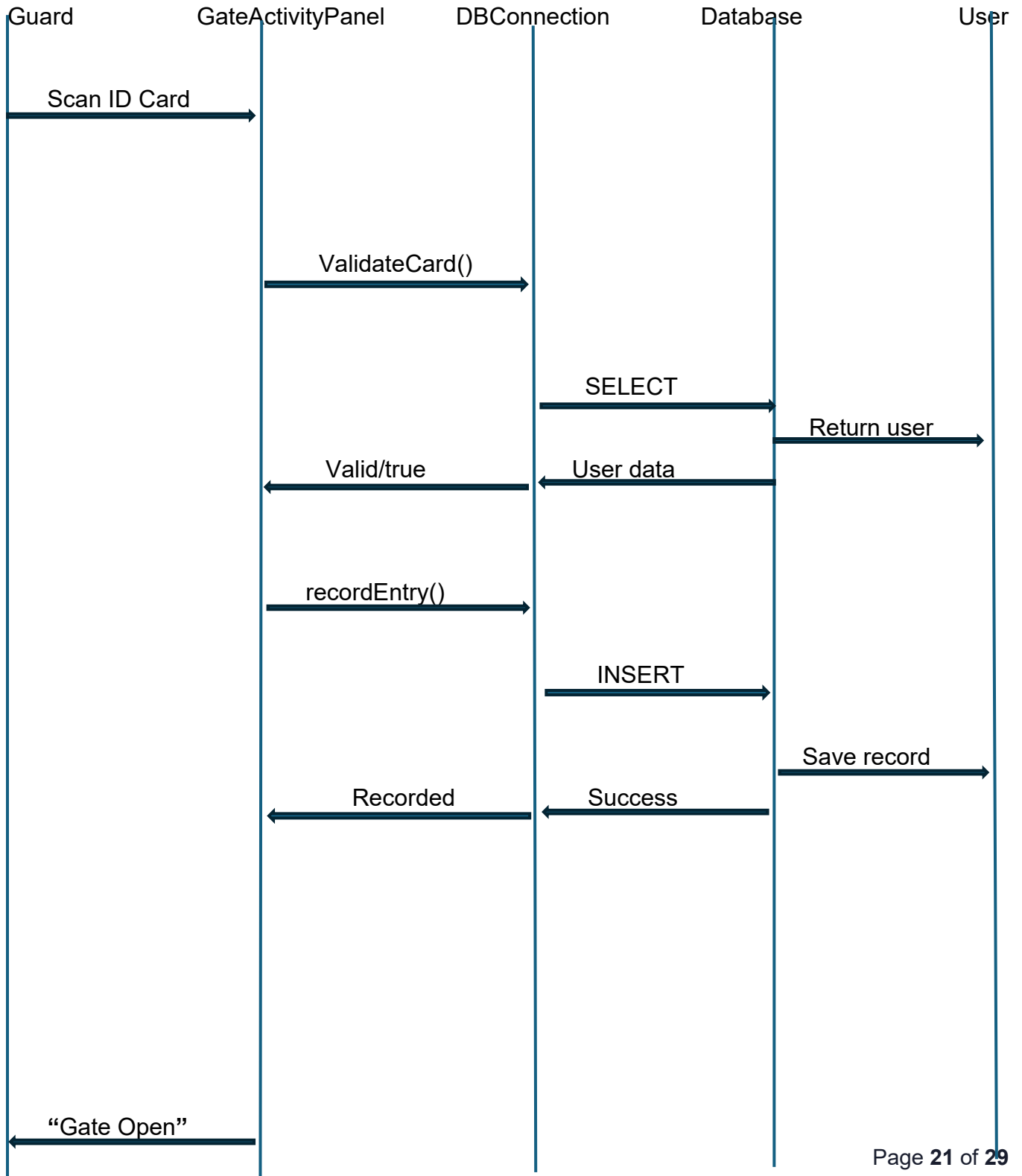
CLASS DIAGRAM

Figure2: Class Diagram for Smart Gate Access Control System

SEQUENCE DIAGRAM(GATE ENTRY FLOW)

Implementation

7.4. Technology Stack Justification

The Smart Gate system has been implemented using a carefully selected technology stack that aligns with the technical capabilities of the development team, the infrastructure available at UNESWA, and the performance and maintainability requirements of the system. The following table provides a comprehensive justification for each technology employed:

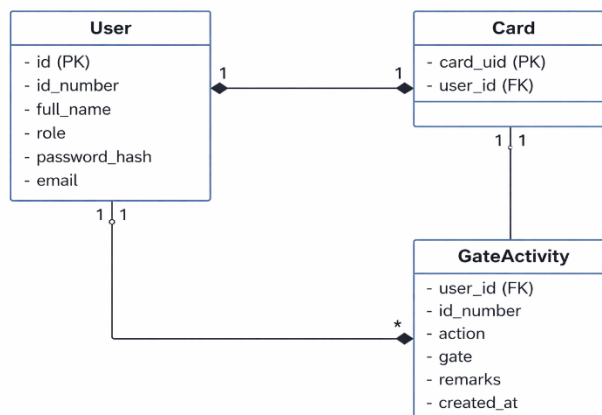
Component	Technology / Tool	Justification
Programming Language	Java SE (JDK 11+)	Platform-independent, strongly typed, widely supported in academic environments and enterprise software.
GUI Framework	Java Swing / JPanel	Native Java UI toolkit; no external dependencies; permits custom-drawn rounded panels, theming, and complex layouts via GridBagLayout and BorderLayout.
Database	MySQL 8.x	Relational DBMS with robust ACID compliance; ideal for structured gate-activity and user-record data; integrates seamlessly with Java via JDBC.
DB Connectivity	JDBC (Java Database Connectivity)	Standard Java API for database interactions; eliminates vendor lock-in and provides PreparedStatement support to prevent SQL injection.
Barcode/QR Scanning	ZXing (Zebra Crossing) Library	Open-source, Java-native library for decoding 1D/2D barcodes and QR codes; supports student/staff ID card scanning without additional hardware drivers.

Component	Technology / Tool	Justification
Build Tool	Apache Maven / Manual JARs	Manages library dependencies (ZXing, MySQL Connector/J) and builds reproducible JAR artefacts for deployment.
Version Control	Git / GitHub	Enables collaborative development, branching, code review, and change tracking across the development team.
Authentication	Role-Based Access Control (RBAC)	Distinguishes Admin, Staff, and Student roles; restricts dashboard access to authorised personnel only.
Deployment	Standalone JAR / Windows EXE Wrapper	Packages the application as a self-contained executable suitable for installation on gate workstations.

7.5. Database Schema

The Smart Gate MySQL database (smart_gate_db) comprises the following principal tables:

- users: Stores user credentials and profile information, including student_id (PK), id_number, full_name, role (STUDENT / EMPLOYEE / ADMIN), password_hash, and email.
- cards: Stores card-to-user associations, with card_uid (PK) linked to user_id (FK referencing users.id). Supports one-to-one card-to-user binding.
- gate_activity: The core audit log table, recording user_id (FK), id_number, action (ENTRY/EXIT), gate (Gate A/B), remarks, and created_at timestamp for every gate event.



The schema is normalised to Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity across tables. Foreign key constraints are enforced at the database level to prevent orphaned records.

7.6. Implementation Environment

- a) Operating System: Windows 10 / 11 (development); Ubuntu Server 22.04 LTS (deployment server)
- b) IDE: IntelliJ IDEA Community Edition with Java 11 SDK
- c) Database: MySQL 8.0.x, administered via MySQL Workbench
- d) Build: Apache Maven 3.9.x for dependency management (ZXing core, MySQL Connector/Jar)

8. Testing

8.1. Testing Strategy

The Smart Gate project adopts a multi-level testing strategy encompassing unit testing, integration testing, system testing, and user acceptance testing. This approach ensures that defects are identified and resolved at the earliest possible stage of the development lifecycle, reducing the cost and effort associated with late-stage fault correction.

8.2. Unit Testing

Unit tests are written for all core business logic components using the JUnit 5 testing framework. Key unit test scenarios include validation of the DB Connection factory method, correctness of JDBC Prepared Statement parameterisation, accuracy of RBAC role assignment logic, and correct parsing of Z-Xing barcode scan results.

8.3. Integration Testing

Integration tests verify that the Presentation, Business Logic, and Data Access layers interact correctly. Key integration test scenarios include the end-to-end gate entry workflow (card scan → Prepared Statement insert → table refresh), the login authentication workflow (credential input → database query → session initialisation), and report generation (database query → JTable population).

8.4. System Testing

System testing is conducted on the fully integrated Smart Gate application in a test environment mirroring the production deployment. Test cases verify all 47 functional and non-functional requirements defined in Section 9, including performance benchmarking of scan response times under simulated load conditions.

8.5. User Acceptance Testing (UAT)

UAT is conducted with a representative sample of end users, including two gate security guards, five students, and three staff members, under realistic operating conditions. UAT sessions are structured around predefined test scripts covering card scanning, manual ID entry, dashboard navigation, and report generation. Acceptance criteria require a task success rate of 90% or above and a System Usability Scale (SUS) score of at least 70 out of 100.

8.6. Testing Schedule

- a) Unit Testing: Ongoing throughout Tasks C, D, E, and F (Weeks 3–7)
- b) Integration Testing: Task H — System Integration and Testing (Week 8)
- c) System Testing: Task H and early Task I (Weeks 8–9)
- d) User Acceptance Testing: Task I (Week 9)
- e) Regression Testing: Prior to deployment in Task J (Week 10)

9. Resources

The following table enumerates all hardware, software, and human resources required for the successful execution of the Smart Gate project. Resource availability has been confirmed with the relevant university departments and suppliers prior to project commencement.

Resource	Type	Quantity	Purpose
Development Workstations	Hardware	4	Java development, testing, and database management
USB Barcode / QR Card Scanners	Hardware	3	Z-Xing-compatible scanners for gate simulation and testing

Resource	Type	Quantity	Purpose
MySQL Server (localhost / VM)	Software	1	Backend database for users, cards, and gate activities
JDK 11+ (Java Development Kit)	Software	All devs	Compilation and execution of Java application
ZXing Core Library (JAR)	Software	1	Barcode and QR code decoding for card scanning
MySQL Connector/J (JDBC Driver)	Software	1	Java-to-MySQL database connectivity
Apache Maven / IntelliJ IDEA	Software	All devs	Dependency management, build automation, and IDE
Git / GitHub Repository	Software	1 repo	Source code version control and team collaboration
UNESWA Test Environment Server	Infrastructure	1	Pre-production testing and UAT deployment
Project Manager Time	Human Resource	36 days	Planning, coordination, reporting, and stakeholder management

10. Quality Assurance

Quality assurance for the Smart Gate project is governed by a structured quality plan that specifies measurable quality attributes, verification criteria, verification methods, and the party responsible for each quality dimension. The quality plan is aligned with ISO/IEC 25010 software quality characteristics, including functional suitability, performance efficiency, usability, reliability, security, and maintainability.

Quality Attribute	Measure / Standard	Verification Method	Responsible Party
Functional Correctness	All 47 functional test cases pass	JUnit unit tests + integration test suite	QA Engineer
Security & Access Control	Zero unauthorised access events in UAT	Penetration testing, RBAC role-switching tests	QA Engineer, Backend Dev.
Response Time	Card scan to gate response < 2 seconds	Stopwatch timing during load testing (50 concurrent)	QA Engineer
Data Integrity	Zero data loss events; ACID compliance	Transaction rollback tests; DB consistency checks	DBA
Usability	≥ 90% task success rate in user testing	Usability study with 10 student/staff participants	Analyst, QA
Code Quality	No critical Sonar Lint/check style violations	Static analysis on all Java source files	All Developers
Availability	System uptime ≥ 99% during operating hours	Stress testing; exception handling review	Backend Dev., DBA

Quality reviews are conducted at each project milestone, with formal sign-off required from the Project Manager and academic supervisor before progression to the subsequent phase. Defect tracking is maintained in a shared GitHub Issues register, categorised by severity (Critical, Major, Minor) and assigned to the responsible developer for resolution.

11. Change Management

Changes to the Smart Gate project scope, schedule, or technical specifications are inevitable in any real-world software development endeavour. A formal change control process has been established to ensure

that all changes are evaluated, approved, documented, and implemented in a controlled and traceable manner.

11.1. Change Control Process

a) Step 1: Change Request Submission

Any team member or stakeholder may submit a Change Request (CR) form to the Project Manager, describing the proposed change, its rationale, and the anticipated impact on scope, schedule, cost, and quality.

b) Step 2: Impact Analysis

The Project Manager, in consultation with the relevant technical lead, assesses the impact of the proposed change on all project dimensions. This analysis is documented and presented to the team at the next weekly meeting.

c) Step 3: Approval or Rejection

Changes with an impact of more than two working days on the project schedule require written approval from the academic supervisor. Minor changes (less than two days' impact) may be approved by the Project Manager alone.

d) Step 4: Implementation

Approved changes are implemented via a feature branch in Git, reviewed by at least one other team member, and merged to the main branch following successful testing.

e) Step 5: Documentation Update

All affected project plan documents, UML diagrams, and test cases are updated to reflect the approved change, and the project baseline is revised accordingly.

11.2. Configuration Management

Configuration management for the Smart Gate project will be implemented through Git version control on GitHub. All source code, database schema scripts, configuration files, and documentation artefacts are maintained in the repository.

Semantic versioning (MAJOR.MINOR.PATCH) is applied to all deliverable releases. The project's baseline configuration is formally established at the end of Task B (Architecture and Database Design) and controlled thereafter via the change control process described above.

References

- Cooper, J. (1984). *Software development management planning*. IEEE Transactions on Software Engineering, 10(1), 22–26.
- ISO/IEC. (2021). *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.
- Sommerville, I. (2026). *Software Engineering* (10th ed.). Pearson Education Limited.
- Oracle Corporation. (2022). *Java Code Conventions*. Retrieved from <https://www.oracle.com/java/technologies/javase/codeconventions-introduction.html>
- ZXing Project. (2023). *ZXing (Zebra Crossing) — Multi-format 1D/2D barcode image processing library*. GitHub. Retrieved from <https://github.com/zxing/zxing>
- Visual Paradigm International. (2024). *Practical guide to Gantt charts*. Retrieved from <https://www.visual-paradigm.com/project-management/practical-guide-to-gantt-charts/>